



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

فاز چهارم: بازسازی شبکه با درخت پوشای کمینه‌ی مقید

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگر مقدم، اسما موحدی تبار

فهرست مطالب

۱	مقدمه	۲
۲	عدم کارایی الگوریتم‌های حریصانه‌ی کلاسیک	۲
۱.۲	مثال نقض ساده	۲
۳	الگوریتم شاخه و کران (Branch and Bound)	۳
۱.۳	ساختار درخت جستجو و وضعیت گره‌ها	۳
۲.۳	کران پایین (Lower Bound)	۴
۳.۳	شرایط هرس (Pruning)	۵
۴	نتیجه‌گیری	۵

۱ مقدمه

هدف، یافتن کم‌هزینه‌ترین درخت پوشا (بر اساس هزینه‌ی ساخت) با رعایت محدودیت مجموع هزینه‌ی نگهداری کابل‌های جدید $B \geq$ است. این مسئله به عنوان **درخت پوشای کمینه‌ی مقید (Constrained MST)** شناخته می‌شود.

در این گزارش، ابتدا نشان می‌دهیم که الگوریتم‌های حریصانه‌ی کلاسیک مانند کروسکال و پریم در حضور قید بودجه‌ی نگهداری، دیگر جواب بهینه را تضمین نمی‌کنند. سپس الگوریتم Branch and Bound را برای حل دقیق این مسئله معرفی کرده و ساختار درخت جستجو، کران پایین، کران بالا و شرایط هرس را به‌طور کامل تشریح می‌کنیم.

۲ عدم کارایی الگوریتم‌های حریصانه‌ی کلاسیک

الگوریتم کروسکال با مرتب‌سازی یال‌ها بر اساس هزینه‌ی ساخت، ارزان‌ترین یال‌ها را انتخاب می‌کند تا یک درخت پوشا با کمترین هزینه‌ی ساخت بسازد. اما این انتخاب ممکن است یال‌هایی با هزینه‌ی نگهداری بالا را شامل شود که مجموع نگهداری از B فراتر رود و درخت نامعتبر گردد. در چنین حالتی، ممکن است یک درخت با هزینه‌ی ساخت بالاتر اما نگهداری کمتر وجود داشته باشد که معتبر باشد و جواب بهینه محسوب شود. کروسکال چنین درختی را در نظر نمی‌گیرد.

۱.۲ مثال نقض ساده

فرض کنید $n = 4$ و $B = 4$. یال‌های زیر موجود هستند (هیچ کابل سالمی وجود ندارد):

یال	u	v	(w, m)
e_1	۱	۲	(۲, ۱)
e_2	۲	۳	(۲, ۱)
e_3	۳	۴	(۲, ۱)
e_4	۱	۳	(۱, ۳)
e_5	۲	۴	(۱, ۳)

- **الگوریتم کروسکال:** ابتدا یال‌های e_1, e_2, e_3 را با هزینه‌ی ساخت ۱ انتخاب می‌کند (چون ارزان‌ترین هستند). هزینه‌ی ساخت کل = ۳، اما مجموع هزینه‌ی نگهداری $= ۲ + ۲ + ۲ = ۶ > ۴$ ، بنابراین این درخت نامعتبر است.

کروسکال هیچ راهی برای یافتن درخت معتبر دیگر ندارد و در صورتی که صرفاً به MST اکتفا کند، ممکن است خروجی NO بدهد.

- جواب بهینه: انتخاب یال‌های e_1 (هزینه‌ی ساخت ۱، نگهداری ۲)، e_4 (ساخت ۳، نگهداری ۱) و e_5 (ساخت ۳، نگهداری ۱). این سه یال یک درخت پوشا تشکیل می‌دهند (۱-۲، ۳-۱، ۴-۲). هزینه‌ی ساخت کل $= ۱+۳+۳ = ۷$ ، و مجموع نگهداری $= ۱+۱+۲ = ۴$ ، بنابراین معتبر است.

بنابراین کروسکال جواب بهینه (هزینه‌ی ساخت ۷) را از دست می‌دهد و اگر صرفاً بر اساس MST تصمیم بگیرد، پاسخ نادرست خواهد داد. این مثال نشان می‌دهد که الگوریتم‌های حریصانه‌ی کلاسیک برای مسئله‌ی Constrained MST کافی نیستند و نیاز به روش‌های دقیق‌تری مانند شاخه و حد است.

۳ الگوریتم شاخه و کران (Branch and Bound)

برای یافتن جواب بهینه، از روش شاخه و حد استفاده شده است. این روش تمام فضای حالت را به صورت سیستماتیک جستجو می‌کند، اما با استفاده از کران‌های پایین و بالا، شاخه‌های غیرامیدوارکننده را هرس می‌کند.

۱.۳ ساختار درخت جستجو و وضعیت گره‌ها

هر گره در درخت جستجو، یک وضعیت (State) را نشان می‌دهد که شامل اطلاعات زیر است:

- index: اندیس کابل بعدی که باید درباره‌ی آن تصمیم‌گیری شود.
- buildCost: مجموع هزینه‌ی ساخت کابل‌های انتخاب‌شده تا این گره.
- maintainCost: مجموع هزینه‌ی نگهداری کابل‌های جدید انتخاب‌شده.
- edgeCount: تعداد یال‌های انتخاب‌شده تا این گره.
- dsu: ساختار Union-Find برای جلوگیری از ایجاد چرخه.
- selectedEdges: لیست اندیس کابل‌های انتخاب‌شده.

ریشه درخت:

- index = 0

• $edgeCount = 0, maintainCost = 0, buildCost = 0$

• dsu با n رأس جدا از هم.

نحوه انشعاب (Branching): در هر گره، برای کابل فعلی ($cables[index]$) دو شاخه ایجاد می‌شود:

۱. **شاخه انتخاب (Include):** اگر کابل چرخه ایجاد نکند و محدودیت نگهداری را نقض نکند، آن را انتخاب می‌کنیم. هزینه‌های ساخت و نگهداری به‌روز می‌شوند، $edgeCount$ افزایش می‌یابد، یال به dsu اضافه می‌شود و اندیس کابل در لیست انتخاب‌شده‌ها ذخیره می‌شود.

۲. **شاخه رد (Exclude):** کابل فعلی را رد می‌کنیم و تنها $index$ را افزایش می‌دهیم.

برای کابل‌های سالم ($unharmed == true$)، هزینه‌های ساخت و نگهداری صفر هستند. در صورت عدم ایجاد چرخه، این کابل‌ها اجباراً انتخاب می‌شوند (زیرا رایگان هستند و هیچ ضرری ندارند) و در غیر این صورت رد می‌شوند.

۲.۳ کران پایین (Lower Bound)

برای هر گره، یک کران پایین بر اساس ادامه‌ی الگوریتم کروسکال از وضعیت فعلی محاسبه می‌شود. ایده این است که با فرض چشم‌پوشی از محدودیت نگهداری، کمترین هزینه‌ی ساخت ممکن برای تکمیل درخت را تخمین بزنیم.

تعریف ریاضی:

$$LB(state) = state.buildCost + \text{وضعیت فعلی درخت از تکمیل برای کروسکال}$$

به این صورت که از یال‌های باقی‌مانده (از $index$ به بعد)، با استفاده از DSU فعلی، به ترتیب هزینه‌ی ساخت، یال‌ها را تا رسیدن به $n - 1$ یال انتخاب می‌کنیم. اگر نتوان به $n - 1$ یال رسید، کران پایین برابر بی‌نهایت است.

اثبات اعتبار:

۱. هر جواب کامل از این گره باید حداقل $n - 1 - edgeCount$ یال دیگر انتخاب کند.

۲. کمترین هزینه‌ی ممکن برای انتخاب این یال‌ها (بدون در نظر گرفتن محدودیت نگهداری) با اجرای کروسکال روی یال‌های باقی‌مانده به‌دست می‌آید.

۳. هزینه‌ی کروسکال $\geq$ هزینه‌ی هر جواب شدنی دیگر (چون کروسکال درخت پوشای کمینه را بدون قید نگهداری پیدا می‌کند).

۴. بنابراین $LB(state) \geq$ هزینه هر جواب کامل ممکن از این گره است و هرگز جواب بهینه را هرس نمی‌کند.

در پیاده‌سازی، تابع `lowerBound` با کپی‌کردن `DSU` فعلی و اضافه کردن یال‌های باقی‌مانده بر اساس هزینه ساخت، این تخمین را محاسبه می‌کند.

۳.۳ شرایط هرس (Pruning)

در هر گره، قبل از انشعاب، چهار نوع هرس اعمال می‌شود:

۱. هرس بودجه نگهداری: اگر $state.maintainCost > B$ ، این شاخه غیرمجاز است و ادامه نمی‌یابد.
 ۲. هرس تعداد یال: اگر تعداد یال‌های باقی‌مانده برای تکمیل درخت کافی نباشد ($e - state.index < (n - 1) - state.edgeCount$)، شاخه هرس می‌شود.
 ۳. هرس هزینه ساخت: اگر $state.buildCost \geq bestCost$ ، این شاخه نمی‌تواند جواب بهتری از جواب فعلی داشته باشد.
 ۴. هرس کران پایین: اگر $lowerBound(state) \geq bestCost$ ، حتی بهترین حالت ممکن از این گره نیز از جواب فعلی بدتر است، پس هرس می‌شود.
- ترتیب اعمال هرس‌ها به گونه‌ای است که ابتدا هرس‌های سریع (بودجه و تعداد یال) و سپس هرس‌های هزینه‌ای انجام می‌شوند تا کارایی افزایش یابد.

۴ نتیجه‌گیری

در مجموع، استفاده از الگوریتم شاخه و کران باعث شد فضای جستجوی نمایی مسئله با استفاده از هرس شاخه‌های غیرامیدبخش به طور قابل توجهی کاهش یابد. هرچند پیچیدگی الگوریتم در بدترین حالت همچنان نمایی است، اما کران پایین مبتنی بر الگوریتم کروسکال و شرایط هرس تعریف‌شده موجب شدند در عمل بسیاری از حالت‌های غیرضروری بررسی نشوند و جواب بهینه با زمان مناسب به دست آید.